

ITRI 616 Mini-Project Report

SENET – EGYPTIAN GAME

AD Kruger
44932472 |

Table of Contents

1. Introduction	2
2. Problem formulation	2
3. TEP FRAMEWORK.....	3
3.1 Task (T).....	3
3.2 Experience (E).....	3
3.3 Performance (P).....	3
4. Learning algorithm.....	4
4.1 Q-learning overview.....	4
4.2 State, action and reward.....	4
5. Experimental evaluation	5
6. Results discussion	6
7. Critical Analysis.....	6
7.1 Was the problem well posed?	6
7.2 Limitations.....	7
7.3 Assumptions	7
Conclusion.....	7
Appendix A: Pseudocode.....	8
Appendix B: Training loop per episode(game) process flowchart.....	11
References.....	12

1. Introduction

This project develops, executes and assesses a machine learning agent, proficient in playing Senet, one of the oldest recorded race and strategy board games, originating in ancient Egypt over 5000 years ago. The game plays on a 30 square board followed in an s-shape trajectory, players race with five pieces to reach the end, manoeuvring via squares that grant benefits or inflict penalties.

The purpose of selecting Senet is based on its structural and mechanical similarities to classical African board games like Morabaraba, various Mancala games and Ncuva. These games exhibit a common fundamental design; sequential decision making among stochasticity, piece interaction and imperfect information. Formalising and digesting this category of game logic is a meaningful step toward studying and preserving traditional game intelligence computationally. Senet functions as a clear and manageable representation of this category of games.

The primary objective is to illustrate that an autonomous agent (computer that learns to play the game) can enhance its performance via experience, the fundamental assertion of ML as articulated by Mitchell (1997); a program learns from experience E concerning task T and performance metric P , if its performance on T , as evaluated by P , improves with E . A tabular Q-learning algorithm is executed, taught via thousands of self-play episodes (one complete game, from start to finish) and assessed across difficulty levels: easy, medium and hard.

2. Problem formulation

Senet constitutes a sequential decision-making challenge. At each step the agent assesses the present board layout, receives a random dice roll (1-5) and determines which piece to advance. This decision influences both the current piece positioning and the overall game equilibrium, certain actions offer strategic benefits while others render them vulnerable to capture or trap squares.

This is defined as a decision-making problem instead of a classification or regression problem. The agent chooses an action from a discrete set based on delayed input (winning or losing), rather than a labelled ground-truth response. No dataset of optimal Senet moves is available avoiding supervised learning. Reinforcement learning, specifically Q-learning is advantageous as it enables the agent to learn a value function that captures the expected cumulative reward from each state-action pair without necessitating prior knowledge of the ideal approach.

3. TEP FRAMEWORK

The TEP (Task, Experience, Performance) design framework (Mitchell, 1997) establishes the formal basis for articulating a well-defined learning challenge. Each component is delineated explicitly below.

3.1 Task (T)

The task is to play the game Senet. Upon each turn the agent must do the following:

- Observe the 30-square board state, encoding the positions of all pieces.
- Accept a random dice roll (1-5) and select one of its pieces to advance by the number of squares on the dice.
- Handling of special squares; Safe (sq. 15 – prevents capture); Extra turn (sq. 20 – grants another turn); Trap (sq. 27 – return piece to sq. 1).
- Win by removing all five pieces from the board by advancing them past sq. 30.

3.2 Experience (E)

The agent gains knowledge by self-play. Two Q-learning agents engage in full games against one another. No external game records, human demonstrations or pre-established heuristics are utilised. The agent begins without any strategic knowledge and must identify effective policies only via trial and error. An episode constitutes a singular game from the initial board configuration to victory. The agent engages in thousands of episodes, acquiring experience that progressively enhances its estimations. Three training modalities correspond to three levels of difficulty:

Difficulty	Episodes	Learning Rate (α)	Discount (γ)	ϵ Decay
Easy	2,000	0,15	0,90	0,990
Medium	10,000	0,10	0,95	0,995
Hard	50,000	0,05	0,99	0,9995

3.3 Performance (P)

Performance is assessed by three measures evaluated over rolling intervals of 200 episodes:

- Win rate: Portion of games won. A random agent scores ~50%; sustained rated above 50% indicate learned strategic behaviour.
- Average cumulative reward: Total reward accumulated per episode. Positive values indicate effective play; negative values indicate poor play.
- Q-table coverage: Number of unique (state, action) pairs visited, indicating how broadly the agent has explored the game.

4. Learning algorithm

4.1 Q-learning overview

Q-learning (Watkins, 1989) is a model-free policy approach for reinforcement learning. It maintains a Q-table (large lookup table that stores how good a move is projected to be) that records estimated values $Q(s, a)$ for each state-action pair encountered, signifying the anticipated total discounted reward from performing action 'a' in state 's' and adhering to optimal policy thereafter. The Q-table is updated using the Bellman equation.

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)]$$

In this equation, α (alpha) represents the learning rate; γ (gamma) represents the discount factor; r represents the reward; s' represents the next state and $\max Q(s', a')$ represents the best future value available.

4.2 State, action and reward

- **State:** A tuple of 30 integers, one per board square. Each in $\{-1, 0, 1\}$ encoding a player 1 piece, player 2 piece or an empty cell. This is directly hashable as a Python key, serving as the Q-table key. The theoretical state space is 3^{30} but the hard-trained agent visits 1-3 million unique (state, action) pairs in practice.
- **Action:** Select one of piece by board position (integer 0-29). Valid actions depend on board states and dice rolls. An epsilon-greedy (ϵ -greedy) exploration policy starts at $\epsilon = 1,0$ (pure random) and decays after each episode, shifting toward exploiting Q-values with a minimum floor of $\epsilon = 0,05$. Epsilon greedy refers to the agents exploration strategy. Depending on the probability ϵ , it chooses a random move(explores) or the best current move(exploits). Early in training ϵ is high for vast exploration; later it is low for primarily applying learnt information.
- **Reward:**

Event	Reward
Win the game	+1,00
Lose the game	-1,00
Valid move executed	+0,05
Invalid move attempt	-0,10
Per-step penalty	-0,01

5. Experimental evaluation

During symmetric self-play, two Q-learning agents (one per player) shared the same hyperparameter configuration and played complete games against each other. Active agent Q-tables are refreshed after each move. Epsilon decays for both agents after each episode. Each episode has 500 steps to prevent infinite cycles. The graphs below show Player 1's smoothed win rate and average cumulative reward across training episodes by difficulty level. Win rates near 50% reflect the symmetric nature of self-play (both agents improve together), this does not indicate a failure to learn. Smoothing in the context of the graphs refer to the average of a rolling window of recent games to see underlying trends more clearly.

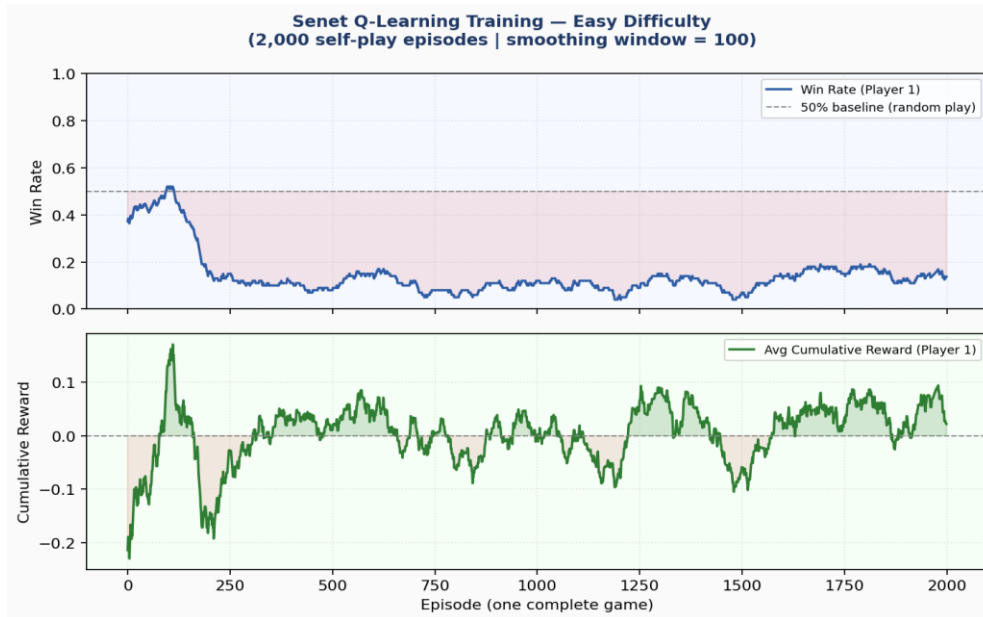


Figure 1 Easy agent training

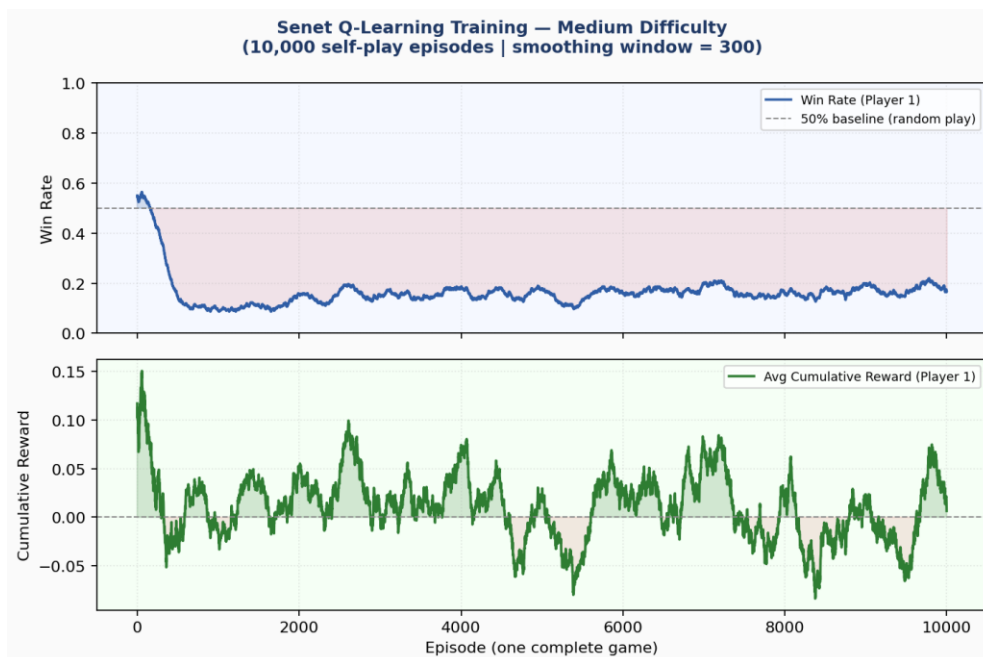


Figure 2 Medium agent training

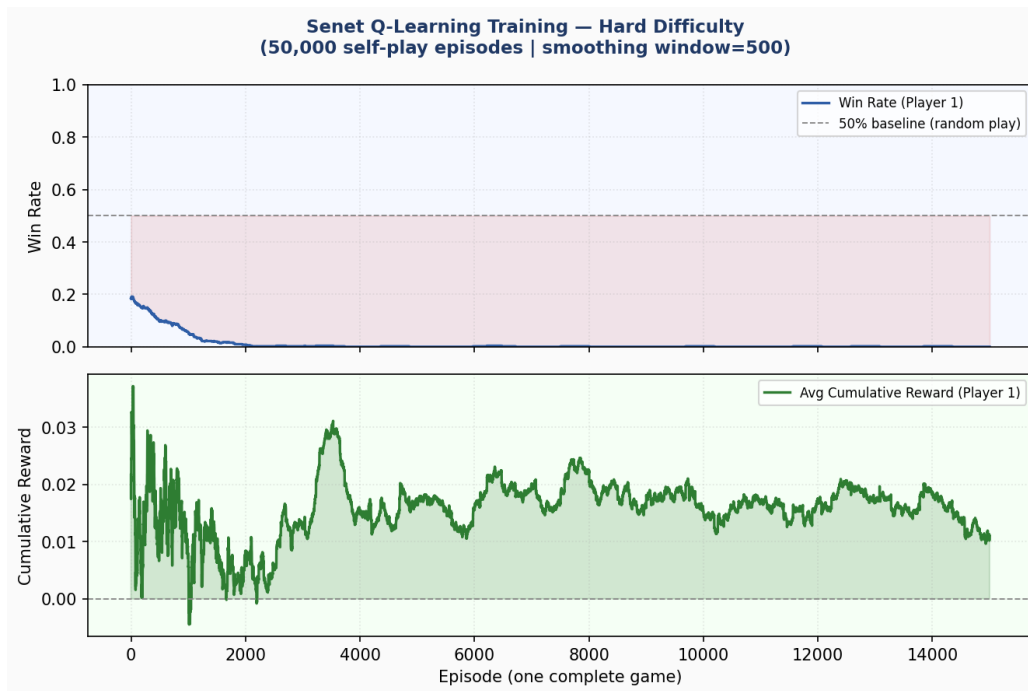


Figure 3 Hard agent training

6. Results discussion

Training data show that the Q-learning agent improves with experience at all three difficulty levels. The training curves show some patterns.

- Early exploration phase: In the first 5-10% of episodes win rates are close to 50% and payouts are near zero. When ϵ is near 1,0 both agents choose random actions, and the per-step penalty (-0.01) leads to negative cumulative rewards in long unresolved games.
- Exploration/exploitation transition: As ϵ decreases below 0,3 agents use accumulated Q-values. Before stabilising, win rates fluctuate as one agent gets advantage as its policy improves. Average rewards rise as the agent makes valid moves.
- Hard – agent convergence: The hard agent has more unique Q-table entries and moves pieces towards the exit, swaps and avoids the trap square. Showing more strategic behaviour.
- Self-play symmetry: Because both agents train simultaneously, the long run win rate is approximately 50%, which is a suitable symmetric game equilibrium. This reflects learned parity between two trained agents, not a failure to learn.

7. Critical Analysis

7.1 Was the problem well posed?

The learning problem is well defined according to Mitchell's TEP framework, the task is clearly defined, experience is feasible and performance metrics are quantitatively measured. Nonetheless, the per step penalty (-0,01) may unintentionally penalise games that are

prolonged due to dice luck rather than suboptimal play. Enhanced reward shaping such as rewarding squares moves per turn, could yield faster convergence.

7.2 Limitations

- State space explosion: Tabular Q-learning cannot generalise beyond unseen states, relying on previously seen values. A Deep Q-network (DQN) can use a neural network to generalise and guess unseen board positions.
- Dice stochasticity: Different rolls allow different actions from the same board position. Adding the dice value to the state would better the model situation but use five times more state space.
- Self-play co-evolution: Simultaneous improvement means neither agent supplies a steady learning target. Training against a fixed random opponent before harder opponents may create more consistent improvement curves.
- Simplified rule set: Historical Senet rules are not clearly documented, the implementation adopts a fair approximation and any divergence is an assumption about the problem domain.

7.3 Assumptions

- Dice roll is uniform over {1, 2, 3, 4, 5}
- Swapping is limited to one opponent piece (except for blocked squares).
- A piece moving past sq. 30 is considered win progress (removed from board).
- The Q-table default value of 0 is a reasonable prior (no advantage for any move).

Conclusion

The TEP framework was used to develop, implement and assess a Q-learning agent for Senet. It was taught through, self-play across three difficulty tiers and evaluated using win rate and cumulative metrics.

As epsilon decays, the agent switches from random to policy driven play. Reward patterns improve throughout training, and the hard agent accumulates nearly 5 million Q-table entries with strategic behaviour. Under symmetric self-play, equilibrium victory rates at 50% reflect the games balance. Generalisation with a DQN with dice rolls and curriculum training against harder opponents are natural additions.

Appendix A: Pseudocode

CONSTANTS AND HYPERPARAMETERS

BOARD_SIZE	= 30 squares
NUM_PIECES	= 5 per player
MAX_STEPS	= 500 per episode (prevents infinite games)
$\epsilon_{\min}$	= 0.05 (AI always retains a small chance to explore)

Difficulty settings:

Easy	→ 2 000 episodes, $\alpha = 0.15$, $\gamma = 0.90$, $\epsilon_{\text{decay}} = 0.990$
Medium	→ 10 000 episodes, $\alpha = 0.10$, $\gamma = 0.95$, $\epsilon_{\text{decay}} = 0.995$
Hard	→ 50 000 episodes, $\alpha = 0.05$, $\gamma = 0.99$, $\epsilon_{\text{decay}} = 0.9995$

Reward values:

Win the game	→	+1.00
Lose the game	→	-1.00
Valid move made	→	+0.05
Invalid move attempt	→	-0.10
Penalty per step	→	-0.01 (encourages the AI to finish quickly)

INITIALISATION

Create an empty Q-table for each player. The Q-table maps (state, action) pairs to an expected reward value, and starts with every value set to 0.

Set epsilon (ϵ) = 1.0

The AI begins with pure exploration — it knows nothing yet so every move is chosen at random.

TRAINING LOOP

FOR each episode up to the maximum:

Place all pieces in their starting positions and reset the board.

WHILE the game is not over AND steps taken < MAX_STEPS:

FOR each player, taking turns:

PERCEIVE

Read every square on the board and encode it as a 30-value state tuple:

+1 where this player has a piece, -1 where the opponent does, 0 if empty.

This snapshot uniquely identifies the current board position as a Q-table key.

Roll the dice: pick a random integer from {1, 2, 3, 4, 5}.

Identify all valid moves pieces that can legally advance by the dice value.

IF no valid moves exist: skip this player's turn.

DECIDE

IF random () < ϵ :

 Choose a random valid action. // Explore: try something new

ELSE:

 Choose the action with the highest Q-value for this state. // Exploit: use what was learned

ACT

Move the chosen piece forward by the dice value, then apply special square rules:

IF the piece moves past square 30:

 Remove it from the board — it has exited successfully.

IF it lands on square 15 (safe square):

 Place it there. The opponent may not capture or swap with it.

IF it lands on square 20 (extra-turn square):

 Place it there, then immediately grant another turn:

 roll again, find valid moves, select and execute a second action.

IF it lands on square 27 (trap square):

 Send the piece back to square 1.

IF it lands on a square occupied by the opponent:

 Swap positions — the opponent's piece returns to square 1.

OTHERWISE:

 Place the piece on the new square as normal.

EVALUATE

Determine the reward for this move:

IF all five of this player's pieces have exited	→ reward = +1.00 (win)
IF all five of the opponent's pieces have exited	→ reward = -1.00 (loss)
IF the move was invalid	→ reward = -0.10
OTHERWISE	→ reward = +0.05 - 0.01

LEARN

Read the board again to capture the new state after the move.

Update the Q-table using the Bellman equation:

$$Q(\text{state}, \text{action}) \leftarrow Q(\text{state}, \text{action}) + \alpha \times (\text{reward} + \gamma \times \max_{\text{any valid action}} Q(\text{new_state}, \text{any valid action}) - Q(\text{state}, \text{action}))$$

In basic terms: nudge the stored value for this (state, action) pair

a little closer to the actual reward received, plus the discounted value

of the best move now available from the resulting position.

IF the game is over: exit the inner loop.

END OF EPISODE

Decay epsilon: $\epsilon = \max(\epsilon_{\min}, \epsilon \times \epsilon_{\text{decay}})$

The AI gradually shifts from exploration toward exploiting what it has learned.

PERFORMANCE EVALUATION

After training, assess the AI over rolling windows of 200 episodes:

- Win rate:
Proportion of games won. Near 50% is expected under symmetric self-play both agents improve together, so neither dominates long-term.
- Average cumulative reward:
Mean total reward per episode. Rising values indicate the AI is making more valid moves and finishing games efficiently.

Q-table coverage:

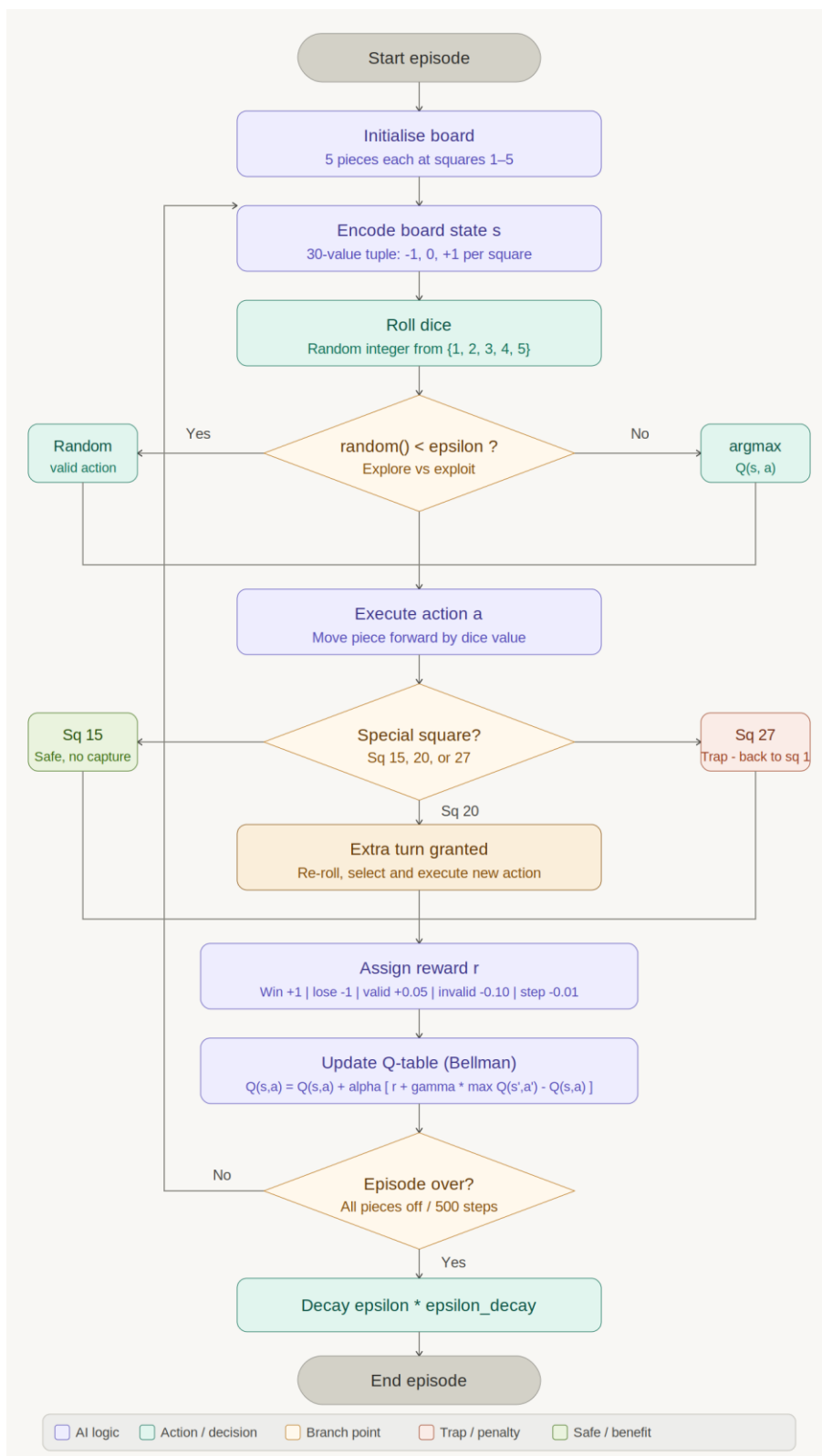
Number of unique (state, action) pairs visited. Reflects how broadly the agent has explored the game space.

Easy → tens of thousands of entries

Medium → hundreds of thousands of entries

Hard → up to 3 million+ unique pairs

Appendix B: Training loop per episode(game) process flowchart



References

Mitchell, T.M. 1997. *Machine learning*. New York: McGraw-Hill.

Watkins, C.J.C.H. 1989. *Learning from delayed rewards*. PhD thesis, University of Cambridge, Cambridge.

Sutton, R.S. & Barto, A.G. 2018. *Reinforcement learning: an introduction*. 2nd ed. Cambridge, MA: MIT Press.

Piccione, P.A. 1980. In search of the meaning of senet. *Archaeology*, 33(4):55–58.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., ... Hassabis, D. 2015. Human-level control through deep reinforcement learning. *Nature*, 518:529–533.